

Introduction to React.js

What is React, JSX, Overview and Syntax



SoftUni Team
Technical Trainers



SoftUni



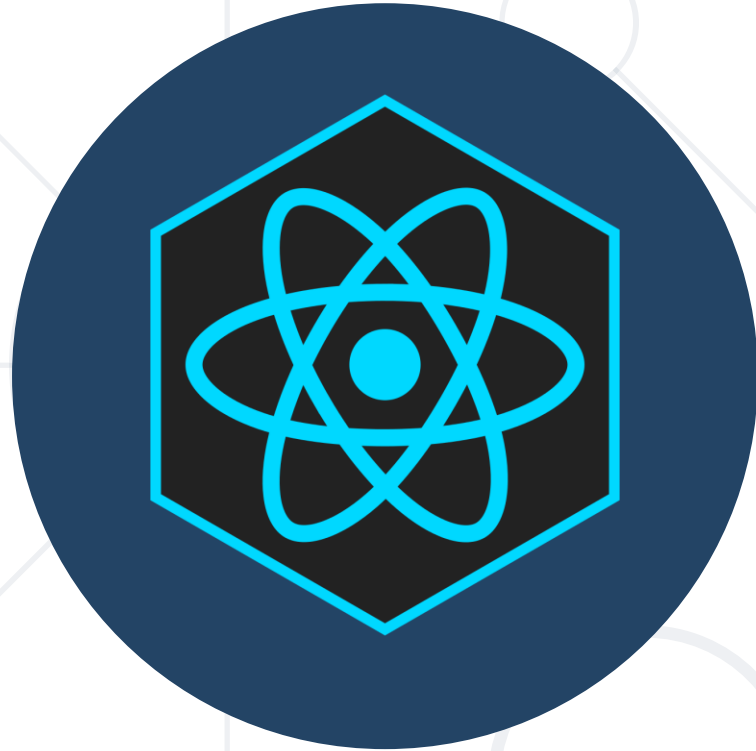
Software University

<https://softuni.bg>

Table of Contents

1. React Overview
2. JSX Syntax
3. Installation
4. Composition
5. React Debugging
6. Components Overview
7. Flow in React
8. Components Props





React Overview

History and Philosophy

History of React.js

- Created By: **Jordan Walke**, a software engineer at **Facebook**
- Release **Timeline**:
 - **2011**: Initial development and deployment on **Facebook's Newsfeed**
 - **2012**: First used on **Instagram**
 - **2013**: Open-sourced at JSConf US (**May 2013**)
- Solve challenges of building **dynamic, high-performance UIs** for large-scale applications
- Simplify data flow with a **component-based architecture**
- Introduce **Virtual DOM** to optimize rendering efficiency

What is React.js?

- **React** is a JavaScript library for building **user interfaces** (UI)
- Focused on creating **reusable components**

```
const HelloMessage = (props) => (  
  <div>Hello {props.name}</div>  
);  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
  
root.render(  
  <HelloMessage name="Maria" />  
);
```

- Open-source
- Declarative
 - Design **simple** views for each **state** in your app
 - Easier to **debug**
- Component-Based
 - Encapsulated **components** that manage their **own** state
 - Keep **state** out of the **DOM**

- Isomorphic
 - JavaScript that runs on **both** client & server
 - Better user experience
- Native support
 - Compose rich **mobile** UI in **Android, iOS**
- Ecosystem
 - Next.js, Remix, Gatsby, Expo, CRA, Vite...



- Fast for development
- High **performance** rendering
- Use all **ES6** features
 - **Promises, Classes** and **Modules**
- Compatible with other **libraries**
- Great **error reporting**





JSX Syntax

Overview, Syntax, Advantages

- **JSX** is React's JavaScript **superset language**
 - Has all of JavaScript's **features** and more
- Unique approach to **mixing HTML and JS**
- Compiles to **plain JavaScript**

```
<div className="red">Children Text</div>
```

```
React.createElement('div',  
  { className: 'red' },  
  'Children Text'  
);
```

- Standard elements use lowercase names
 - **div, span, form, input, ...**
- Custom components **always** use Pascal case
 - **MyCustomComponent, Greeting, ScoreBoard, ...**
- Component name cannot be an expression
 - Use a **variable** instead
- There must be a **root element**
- More info at: <https://react.dev/learn/writing-markup-with-jsx>

- JSX **compiles** to function calls

```
<div className="red">Children Text</div>
```

Element type (HTML tag name)

Properties object

```
React.createElement('div',  
  { className: 'red' },  
  "Children Text" [, ...]);
```

List of children



React Installation

Packages, Setup, Structure

- Instant Feedback - **rapid reloading** for efficient development
- Only one dependency - no complicated version mismatches
- No Lock-In - under the hood **Webpack, Babel, ESLint**



Install and Run Vite

- Run the React app creator

```
npm create vite my-app
```

- Select Framework >> **React**

- Select Variant >> **JavaScript**

- Starts your React app from the command line

```
cd my-app
```

```
npm install
```

```
npm run dev
```

- Browse your app from <http://localhost:5173>

- Visit the **official website**

<https://reactjs.org/>

- Documentation

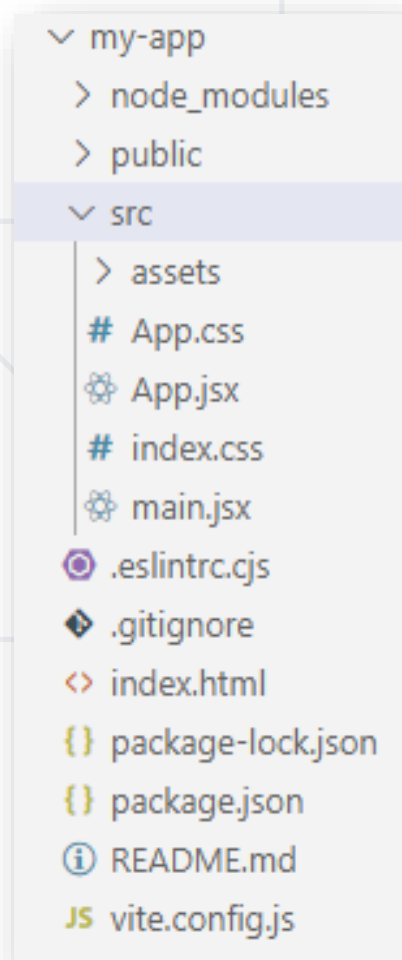
<https://react.dev/learn/installation>

- Online sandbox

<https://codesandbox.io/>

React App Structure

- **package.json** - project configuration
 - Module name, dependencies, build actions
- **index.html**
 - App main HTML file
- **main.jsx**
 - App main JS file (startup script)
- **App.jsx, App.css,**
- React component "**App**"





Components Overview

Syntax, Functional and Class Components

Components Overview

- Components let you
 - Split the UI into **independent** and **reusable pieces**
 - Think about **isolation**
- React let you define components as Functions

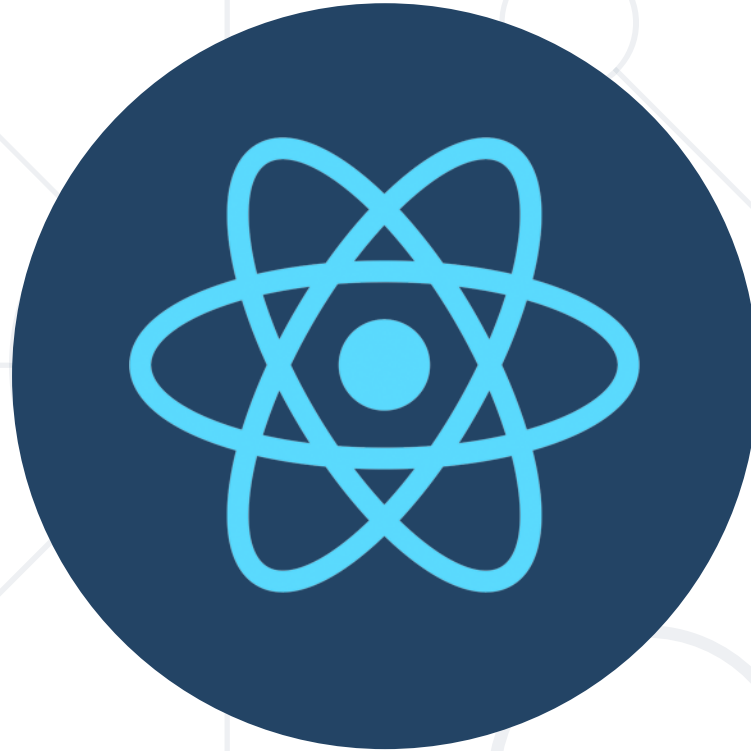


- **Functional component** is a JS function which
 - Accepts single argument called **props** (object with data)
 - Returns a **React Element**

```
function Person(props){  
  return <div>My name is {props.name}</div>  
}
```

- Names always start with **UpperCase**
- Tags always must be **closed**
- **Information** is passed via **props**

```
function SimpleComponent(props) {  
  return (  
    <div>  
      <h1>{props.title}</h1>  
      <p>{props.description}</p>  
    </div>  
  );  
}
```



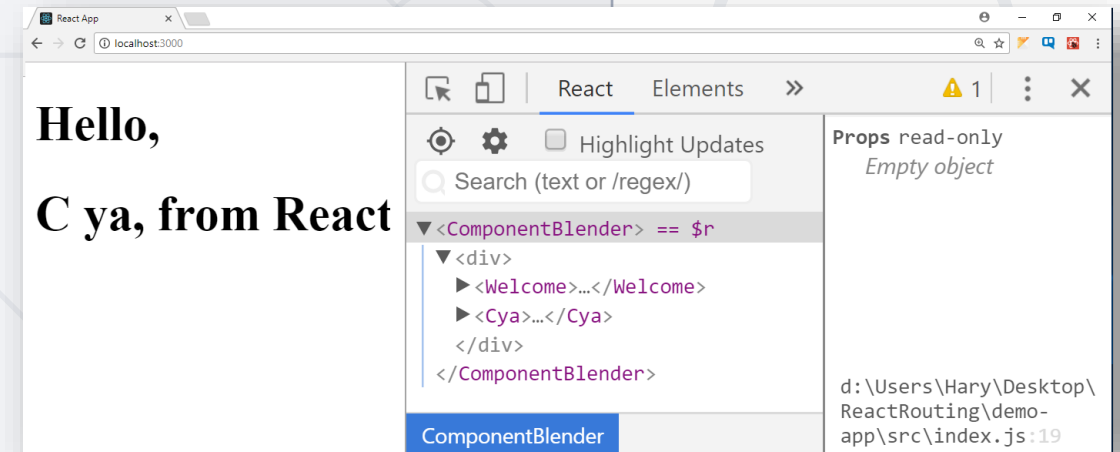
Composition

Definition and Advantages

- React components can be **nested**, like DOM elements

```
function Welcome() {  
  return <h1>Hello, from React</h1>;  
}  
function Cya() {  
  return <h1>C ya, from React</h1>;  
}  
function ComponentBlender() {  
  return (  
    <div>  
      <Welcome />  
      <Cya />  
    </div>  
  );  
}
```

```
ReactDOM.createRoot(  
  document.getElementById('root')  
)  
.render(<ComponentBlender />);
```



- Names always start with **uppercase**
- Tags must be **closed**
 - If there are no children - use **self-closing tags**
- **Information** is passed via **props**

```
<Heading size="large">Hello React</Heading>
```

```
<UserHead name="homeHeader" />
```


- Encapsulate logic
- **Separate** your code
 - Easier to **maintain** and **debug**
 - Allows **reusability**
- Components are neat





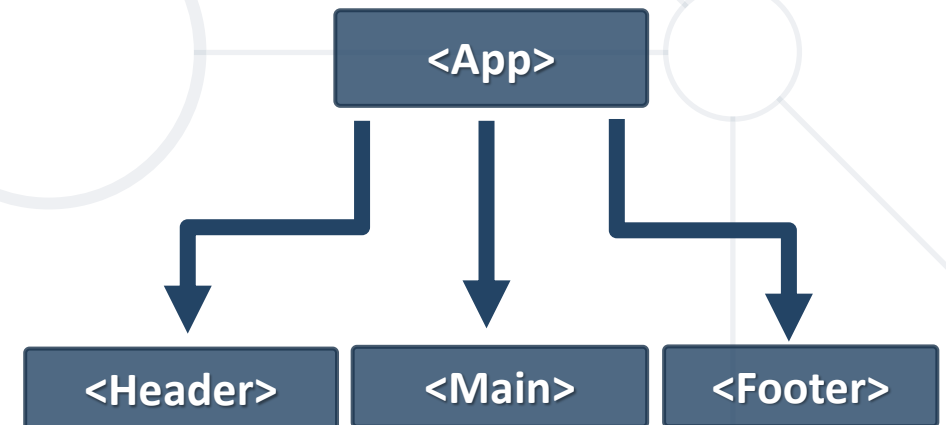
Flow in React

Understanding the Flow in React

- React optimizes rendering by using a structured approach involving **three** key elements:
 - **Component Tree** – The blueprint of the UI, defining how components are structured and interact
 - **Virtual DOM (VDOM)** – A lightweight copy of the actual DOM that helps React efficiently track changes
 - **DOM Tree** – The final representation rendered in the browser, updated selectively by React for better performance

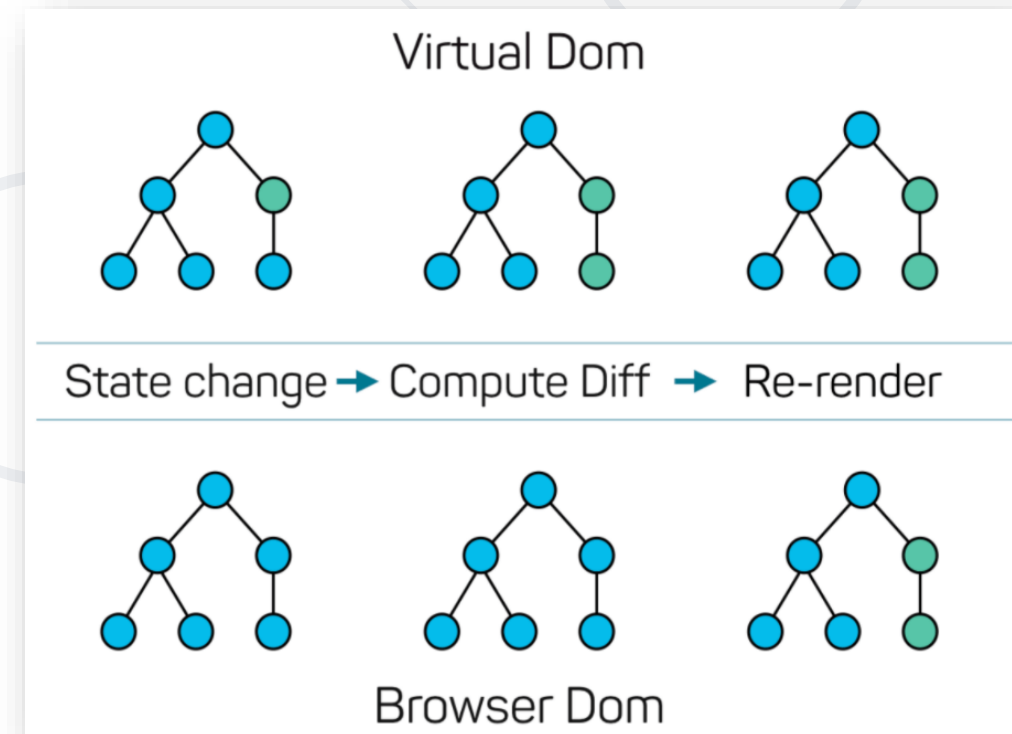
Component Tree (React Components)

- A **hierarchical** structure of React components
- Each component represents a part of the **UI**
- Components have **state & props** to manage data



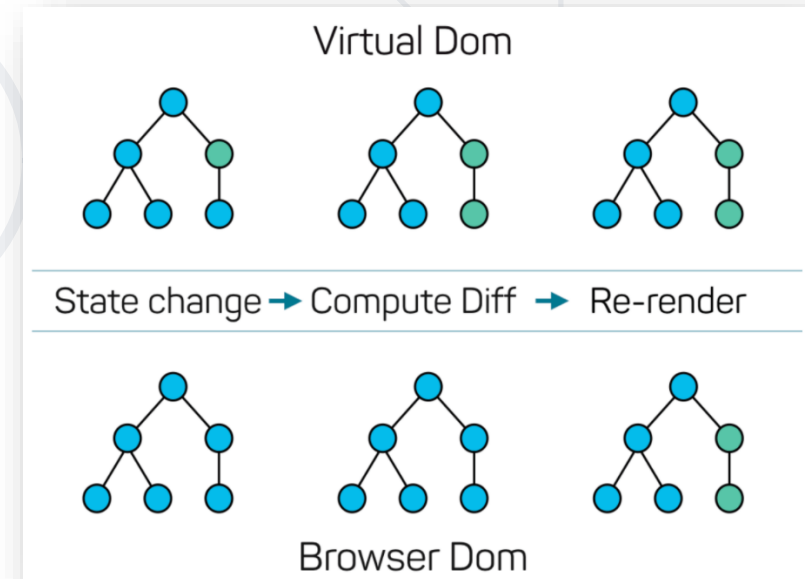
Virtual DOM (VDOM)

- A **lightweight** copy of the actual DOM
- React updates the **VDOM** first instead of modifying the real DOM directly
- **Reconciliation Algorithm** compares changes and updates only the necessary parts of the DOM

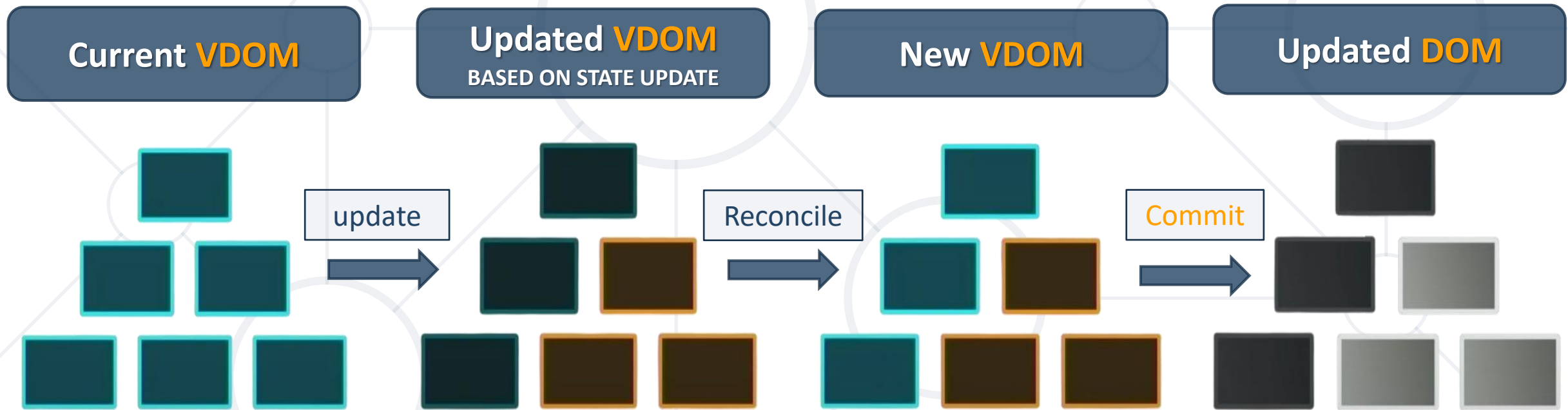


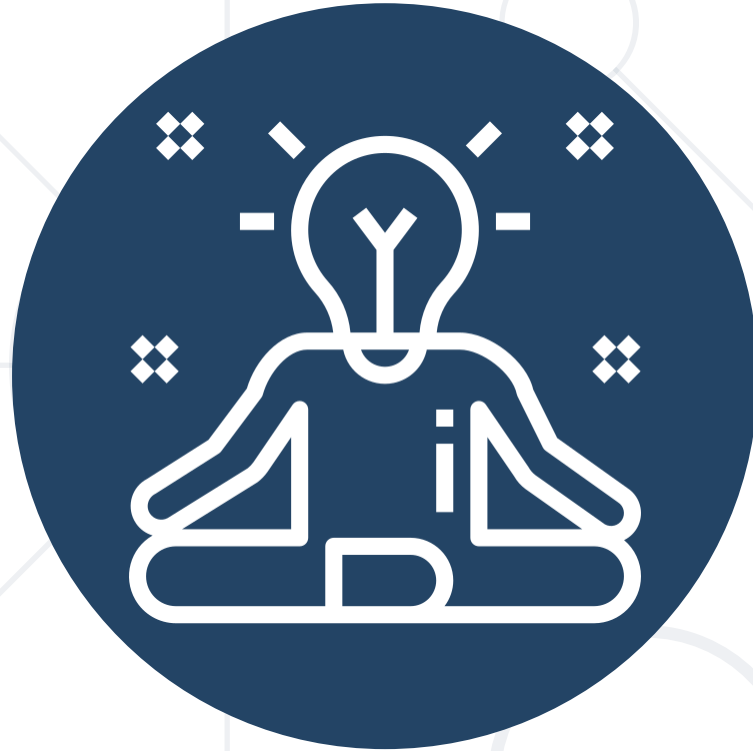
DOM Tree (Real Browser DOM)

- The **actual rendered UI** in the browser
- Directly manipulated by JavaScript, but modifying it is **slow** compared to the VDOM
- React **minimizes DOM updates** by batching changes efficiently



Flow in React





Component Props

Overview

Props and State Overview

- In React **props** and **state** represent the rendered values
- Both are plain JavaScript **objects**
- Both hold information that influences the output of render



Props and State Overview

- They are different in one important way
 - **Props** get passed to the component (like function params)
 - **State** is managed within the component (like local variables)



- **Props** is short for **properties**
 - Are received from above (parent)
 - **Immutable** as far as the component receiving them is concerned
- A component **cannot change** its own props, but it is responsible for putting together the props of its child components

Passing Props to Nested Components

- We use props to **pass data** from parent to child

```
const BookList = () => {  
  return (  
    <ul>  
      <Book  
        title="IT"  
        author="Stephen King"  
        price="20"  
      />  
      <Book  
        title="The Hunger Games"  
        author="Suzanne Collins"  
        price="10"  
      />  
    </ul>  
  );  
};
```

Prop name should start with lowercase letter

Use className to set css classes

```
const Book = (props) => {  
  return (  
    <li className="book">  
      <div>{props.title}</div>  
      <div>{props.author}</div>  
      <div>{props.price}</div>  
    </li>  
  );  
};
```



React Debugging

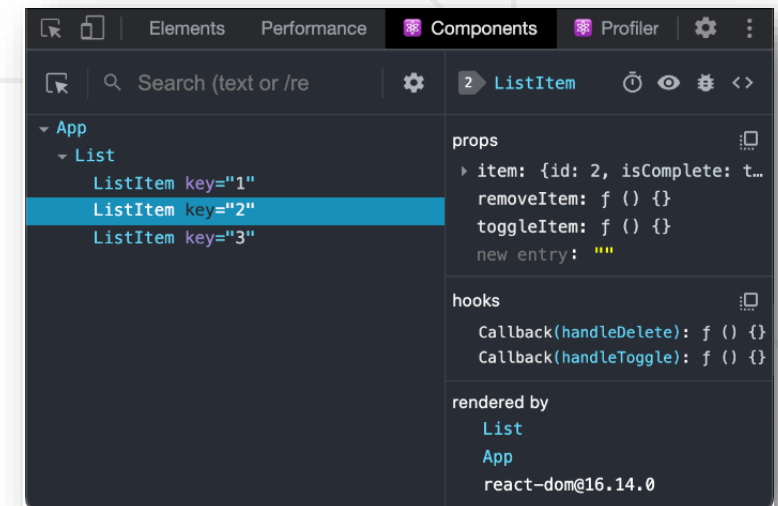
Debugging React Components

- Debugging is a **critical** skill in development. It helps identify and fix issues within your code
- React applications can have **complex** component hierarchies. Debugging ensures components work as expected



Debugging Using React Developer Tools

- The easiest way to debug websites built with React is to install the React Developer Tools **browser extension**
 - It help inspect the React component hierarchy, view props and state, and track component re-renders
 - Here you can find **download** links and more info react.dev/learn/react-developer-tools



- Open your React app project folder in VS Code
- Click on the "**Run and Debug**"
- Select "**Web App (Chrome)**" for debugger
- Modify the generated "**launch.json**" by changing **URL** to match your app address
- Press **F5** button

```
"version": "0.2.0",  
"configurations": [  
  {  
    "type": "chrome",  
    "request": "launch",  
    "name": "Launch Chrome against localhost",  
    "url": "http://localhost:5173",  
    "webRoot": "${workspaceFolder}"  
  }  
]
```


- **React** is a JavaScript library for building user interfaces
- **React** uses all **ES6** features
- **JSX** is React's JavaScript **superset**
- React **components** can be nested (composition)
- **Debugging** in React
- **Components** - reusable elements



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

